method[3] for the class, or even consider restricting its visibility to *private*.

The 2 main operations that we'd like to perform are:

1. Given two elements `x` and `y`, both belonging to `U`, we'd like to check if they belong to different partitions (`areDisjoint`);
2. Given two elements `x` and `y`, we'd like to merge their partitions into a single one (`merge`).

5.4 Naïve Solution[4]

The most immediate solution for our problem is to represent each partition with a list (or array), as illustrated in figure 5.4: for each element, we need to keep track of the pointer to the head of the list.

To find out if two elements are in the same partition, we need to retrieve the list for one element, and check out if it is the same list as for the other[5].

To merge two partitions `P1` and `P2`, modeled with two lists `L1` and `L2`, we need to update the `next` pointer[6] of the last element in `L1` so that it points to the head of `L2` (or vice versa, with the last element of `L2` pointing to the head of `L1`). This can be done in constant time by keeping an extra (constant space) pointer to the tail of each list. Unfortunately though, we wouldn't be done just yet: for every element in `L2`, we also need to update its list pointer in our map to point to the head of the new merged list.